

day 17

day 3 node

📌 Obsidian Notes: Express.js & REST API Architecture

المفهوم الأساسي

بدلاً من كتابة كل كود السيرفر في ملف واحد كبير، نستخدم نظام **MVC (Model-View-Controller)** لتقسيم الكود. الهدف إن كل ملف يكون مسؤول عن حاجة واحدة بس (Separation of Concerns).

1. هيكل المشروع (MVC Directory Structure)

- **index.js**: الـ Entry Point الـ يستقبل الـ Requests السيرفر، بيشتغل الـ **Middlewares** الأولى ويبشغل الـ **Requests** السيرفر، بيستقبل الـ **index.js** الـ **express.json()**.
- **routers/**: الـ مسؤولة بس عن توجيه الـ (GET /users , POST /users زي) الـ **Endpoints** المسارات والـ **Request** الـ **Controller** المناسب.
- **controllers/**: الـ **Logic** الحقيقي (الـ **Logic** قلب التطبيق! هو اللي فيه الـ **Response**).
- **model/**: الـ **JSON** (حالياً ملفات) مكان تخزين البيانات: **users.json** و **posts.json**.
- **middleware/**: الـ **Handlers** الفلاتر والـ **notFoundHandler** والـ **errorHandler** زي الـ **File Operation** ودوال الـ **Custom Error** **AppError** زي الـ **utils/** (الأدوات المساعدة المكررة).

2. طريقة التعامل مع البيانات والتعديل عليها

عشان نعمل **CRUD** كامل على ملفات الـ **JSON**، بنستخدم دالتين أساسيتين:

1. **readFile**: الـ **JSON** بتقرأ ملف الـ **JSON.parse** وبتعمله **Array** أو **Object** عشان نحوله لـ **JSON.stringify** بعد التعديل وبتعملها **Array** بتاخذ الـ **writeFile**.
2. **writeFile**: عشان تحفظها جوه الملف تاني **JSON.stringify** بعد التعديل وبتعملها **Array** بتاخذ الـ **writeFile**.

الفرق بين الـ PUT والـ PATCH:

- **PUT**: كامل بـ **Object** لازم العميل بيعت الـ **(name, age, email)** كامل بـ **Full Replace** تحديث كامل.
 - **PATCH**: نستخدم الـ **(email)** (مثلاً) العميل ببيعته الحق الـ **Partial Update** تحديث جزئي.
- للدمج **Spread Operator**: `{ ...user, ...body }`.

3. طريقة إدارة الأخطاء (Centralized Error Handling)

بدل ما نكتب **try/catch** ونقعد نبعت **res.status().json()** في كل مكان:

1. **AppError (Custom Error Class)**:

كلاس بسيط بيخلينا نحدد الـ Message والـ StatusCode في مكان واحد:

JavaScript

```
throw new AppError("User not found", 404);
```

2. errorHandler Middleware:

ميدل وير بيتخط في آخر `index.js` خالص، وظيفته يلقف أي خطأ يتربى بـ `throw` أو يتبعث لـ `next(err)` ويرجع الـ `Response` بالشكل المضبوط:

JavaScript

```
if (err instanceof AppError) {  
  return res.status(err.statusCode).json({ message: err.message });  
}
```

4. العلاقات بين البيانات (One-to-Many Relationship)

- المفهوم: الـ User الواحد عنده أكثر من (One User -> Many Posts).
- التطبيق العملي:
 - جوه كل Post، بنحط حقل اسمه `userId` بيشير للـ `id` بتاع المستخدم اللي كتبه.
 - التحقق قبل الإضافة: لما نيجي نكريت Post جديد، بنقرأ ملف الـ Users الأول عشان نتأكد إن الـ `userId` ده موجود فعلاً!
 - الحذف المتتابع (Cascading Delete - Bonus): لما نيجي نمسح User، بنقرأ ملف الـ Posts ونمسح كل البوستات اللي الـ `userId` بتاعها بيساوي ID الـ User اللي اتمسح.

ملخص الأوامر والـ Endpoints المطلوبة للتاسك

Method	Endpoint	Description
GET	/users	جلب كل المستخدمين
GET	/users/:id	جلب مستخدم محدد بـ ID
POST	/users	إضافة مستخدم جديد
PUT	/users/:id	تعديل كامل بيانات المستخدم
DELETE	/users/:id	حذف المستخدم (وحذف بوستاته تلقائياً)
GET	/posts	جلب كل البوستات
POST	/posts	إضافة بوست جديد (بشرط وجود الـ userId)

أسئلة إنتر views متوقعة (Interview Q&A) ?

② **Q1: Why do we place `express.json()` middleware before the routers in `index.js` ?** الإجابة: لأن `express.json()` يقوم بعمل **parsing** للـ **Request Body** المبعوث بصيغة JSON، سنكون قيمة **Routers** إذا وضعت بعد الـ `req.body` وتخزينه في **JavaScript Object** وتحويله لـ **JSON** داخل الـ **Controllers**. `req.body` بـ `undefined`

② **Q2: What is the main structural difference between PUT and PATCH HTTP methods?** الإجابة:

- **PUT:** يفترض إرسال المورد بكافة حقوله، وإذا غاب حقل قد يمحي أو (Full Replacement) استبدال كلي للمورد بـ `null` يعوض بـ.
- **PATCH:** يستقبل فقط الحقول المراد تغييرها ويحافظ على بقية البيانات كما (Partial Update) تعديل جزئي للمورد هي.